System Verification and Validation Plan for FitFolio

Team #1, Burn n' Earn
Davis Lenover
Preston Grewal
Angad Chhabra
Farid Bastoros
Zaid Alsaadi

October 30, 2026

Table 1: Revision History

Date	Developer(s)	Change
Oct 26–27, 2026	Davis Lenover	3.6 Automated Testing and Verification Tools.

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 2
- 2.3 Custom Documents . . . 4
- 2.4 Relevant Documentation . . . 4

3 Plan **5**
- 3.1 Verification and Validation Team . . . 5
- 3.2 SRS Verification . . . 5
- 3.3 Design Verification . . . 5
- 3.4 Verification and Validation Plan Verification . . . 5
- 3.5 Implementation Verification . . . 6
- 3.6 Automated Testing and Verification Tools . . . 6
  - 3.6.1 Linting and Formatting . . . 6
  - 3.6.2 Unit and Integration Testing . . . 7
  - 3.6.3 Secret Scanning . . . 8
- 3.7 Software Validation . . . 8

4 System Tests **9**
- 4.1 Tests for Functional Requirements . . . 9
  - 4.1.1 Area of Testing1 . . . 9
  - 4.1.2 Area of Testing2 . . . 10
- 4.2 Tests for Nonfunctional Requirements . . . 10
  - 4.2.1 Area of Testing1 . . . 10
  - 4.2.2 Area of Testing2 . . . 11
- 4.3 Traceability Between Test Cases and Requirements . . . 11

5 Unit Test Description **11**
- 5.1 Unit Testing Scope . . . 12
- 5.2 Tests for Functional Requirements . . . 12
  - 5.2.1 Module 1 . . . 12
  - 5.2.2 Module 2 . . . 13
- 5.3 Tests for Nonfunctional Requirements . . . 13
  - 5.3.1 Module ? . . . 13

5.3.2 Module ? 14
5.4 Traceability Between Test Cases and Modules 14

6 Appendix **15**
6.1 Symbolic Parameters 15
6.2 Usability Survey Questions 15

7 Appendix — Generative AI Use Disclosure **16**
7.1 AI Tools Used 16
7.2 How and Where Used 16
7.3 Verification of AI-Generated Content 16

8 Appendix — Reflection **17**

List of Tables

1 Revision History i

[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS (Author, 2019) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

This section identifies the software under test, states the objectives of verification and validation (V&V) and what is out of scope, summarizes the custom documents, and lists the documents this plan draws on.

2.1 Summary

The software being tested is FitFolio, a mobile application in which young adults earn virtual currency from the physical activity their phone records and invest it in a simulated market that follows real-world prices. FitFolio consists of a mobile app for Android and iOS, built with React Native and TypeScript, and a server built with FastAPI and a PostgreSQL database. The app reads activity from the phone's health platform (Apple HealthKit or Google Health Connect), and the server obtains asset prices from the market data provider, Finnhub. The full system and its context are described in Section 2.3 of the Software Requirements Specification (SRS; Team 1, Burn n' Earn, 2026b).

The functions of FitFolio fall into the six feature areas of SRS Section 6, which also organize the system tests in Section 4 of this plan:

- **Accounts and profile:** creating an account, signing in and out, resetting a password, editing the profile, downloading a copy of one's data, and deleting the account.
- **Activity and rewards:** reading activity records for the supported activity types, converting plausible activity into virtual currency up to a daily cap, and tracking goals, streaks, rest days and milestones.
- **Simulated market and portfolio:** buying and selling whole shares of at least 50 real-world stocks at prices that follow the real market, showing holdings and portfolio value, and practising in market scenarios that replay past periods.

- **Learning and feedback:** explaining financial and fitness concepts where they appear, giving feedback on the user's own trades and portfolio changes, and testing understanding with knowledge checks.
- **Social feed:** adding friends by username and sharing selected statistics and milestones with chosen friends.
- **Reminders and settings:** sending reminders only in the periods the user chooses, and letting the user review and withdraw health data permissions.

Because FitFolio handles health data and presents financial information that users may act on, the plan also verifies the safety and security requirements identified in the Hazard Analysis (HA; Team 1, Burn n' Earn, 2026a), which the SRS restates in Sections 6 and 7.3.

2.2 Objectives

The objectives below are the qualities the team most needs to be confident in, given the time and people available. They are listed in priority order, with the SRS goals (G) and non-functional requirements (NFR) each one comes from.

- **Correct currency, rewards, and portfolio values.** Build confidence that activity earns the right amount of virtual currency, that each activity is rewarded only once, and that balances, trades, and portfolio values are calculated correctly, including when a trade is interrupted or repeated (G4, G5, NFR1 Accuracy, NFR6 Reliability). This comes first because users judge the app by whether its numbers are right, and in an educational app a wrong number also teaches a wrong lesson.
- **Private and secure user data.** Demonstrate that private information, including health data, is available only to authenticated and authorized users, and that nothing is shared with friends unless the user chooses to share it (G3, NFR7 Security). FitFolio handles health data, so a leak would harm users and break the privacy rules it must follow (SRS, SC6).

- **Adequate usability and accessibility.** Validate through usability testing that first-time users can complete the core activity and investing tasks without help, including with enlarged text, and that participants improve on the before-and-after knowledge check (G1, G2, NFR2 Usability, NFR8 Accessibility). The results will be reported in the Usability Report.

- **Core features on Android and iOS.** Demonstrate that the core features work on both supported platforms (NFR4 Portability). Since one React Native codebase targets both, the same acceptance tests are run on each.

The following are out of scope, because the team does not control them or does not have the resources to verify them:

- **External libraries and services.** The phone's health platform (Apple HealthKit or Health Connect), the libraries that connect to it, the market data provider, and frameworks such as FastAPI, SQLAlchemy, and PostgreSQL are assumed to have been verified by their own teams. How FitFolio handles wrong, missing, late, or duplicated data from these services is still in scope (Hazard Analysis, CA4).

- **Accuracy of recorded activity and market prices.** FitFolio rewards the activity the health platform records and values portfolios at the provider's prices. Whether a phone counts steps correctly, or whether a quoted price matches the exchange, cannot be checked by the team. The team verifies how FitFolio uses these values, but not the values themselves.

- **Load and scalability testing.** No requirement covers large numbers of users, and during the project the app will be used only by the team, the supervisor, and usability test participants. Performance is checked only as stated in NFR5.

- **Attacks from a compromised phone or hosting provider.** Security testing covers FitFolio's own authentication and authorization. A rooted or jailbroken phone and the hosting provider's physical security are outside the team's control (Hazard Analysis, CA3 and CA5). No external penetration test is planned.

- **Long-term effects on users.** Whether FitFolio changes users' fitness habits or financial decisions over months cannot be measured within the project. Learning is measured only by the short-term knowledge check in G1.

2.3 Custom Documents

FitFolio has two custom documents (CDs), one in each term, as listed in Section 4 of the Problem Statement and Goals. They are the project's extras, and both feed directly into validation.

Design Thinking Report (Fall). This report explores what young adults want to track about their fitness and what they hope to learn about investing. Its main evidence is two exploratory interviews with potential users, held on October 14, 2026, and referred to as P01 and P02. These interviews are the source of most of the stakeholder requirements in SRS Section 3 and of Personas 1 and 2 in SRS Section 4. For V&V, the report provides the needs that validation checks the product against. It also exposes a gap that validation must close: neither interviewee was a beginner at investing, so the needs of beginners, represented by Persona 3, come from the Problem Statement rather than from users. The usability tests in this plan therefore recruit participants who are beginners at investing (Section 3.7).

Usability Report (Winter). This report documents usability testing of the working application, covering onboarding, earning currency from activity, trading, learning, and sharing. The usability tests specified in this plan are the ones the report will run and record: the participant profile, the tasks, the observations, and the questionnaire are defined here (Section 3.7 and Appendix 6.2), and the report presents their results. These tests produce the evidence for NFR2 (Usability) and for goals G1 (at least 70% of participants improve their score on a before-and-after knowledge check) and G2 (at least 80% of first-time participants complete the core tasks without outside help) from SRS Section 2.2. Because the tests need a working application, they take place after the Rev 0 demonstration.

2.4 Relevant Documentation

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

Author (2019)

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the review. Will you have a meeting? What will you present? What questions will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

3.4 Verification and Validation Plan Verification

[The verification and validation plan is an artifact that should also be verified. Techniques for this include review and mutation testing. —SS]
[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

3.5 Implementation Verification

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

[In this section you would also give any details of any plans for static verification of the implementation. Potential techniques include code walkthroughs, code inspection, static analyzers, etc. —SS]

[The final class presentation in CAS 741 could be used as a code walkthrough. There is also a possibility of using the final presentation (in CAS741) for a partial usability survey. —SS]

3.6 Automated Testing and Verification Tools

Ruff lints and formats the backend's Python code, and pytest runs its unit and integration tests. For the frontend, ESLint and Prettier lint and format the TypeScript code, and Jest runs its unit tests. gitleaks scans the whole repository for secrets. Each runs in GitHub Actions on every pull request and every push to `main`, and the team's workflow requires every check to pass before a pull request is merged (Development Plan, Workflow Plan). The reasons for choosing each tool are given in the Development Plan's Expected Technology section.

3.6.1 Linting and Formatting

Ruff is used to lint and format the backend's Python code, in place of the separate Black and Flake8 tools. It is configured in `src/backend/pyproject.toml` and enforces the team's coding standard (Development Plan, Coding

Standard), including the PEP 8 line length of 79 characters. The enabled rule sets are:

- **E, W:** pycodestyle errors and warnings (PEP 8 style).
- **F:** pyflakes, such as unused imports and undefined names.
- **I:** import order.
- **B:** flake8-bugbear, which flags likely bugs such as mutable default arguments.
- **UP:** pyupgrade, which flags syntax that has a newer equivalent.
- **N:** PEP 8 naming conventions.
- **SIM:** flake8-simplify, which flags code that can be written more simply.

The `Lint (Ruff)` job in `.github/workflows/backend-ci.yml` runs `ruff check` and `ruff format --check` on both the backend source and its tests in `test/backend`. The job fails if any rule is broken or any file is not formatted, so unformatted code cannot reach `main`.

The frontend's TypeScript code will be linted with ESLint, which catches common mistakes, and formatted with Prettier, configured to use four spaces for indentation to match the coding standard. Both will run in CI on every pull request and fail the check if any rule is broken or any file is not formatted, in the same way as Ruff.

3.6.2 Unit and Integration Testing

pytest is used for the backend's automated tests, which live in `test/backend`. The `Test (pytest + PostgreSQL)` job in `.github/workflows/backend-ci.yml` starts a PostgreSQL 18 container, applies the database migrations with Alembic, and then runs the full test suite. Tests that read or write data therefore run against a real database with the same schema the app uses, rather than a mock. The CI database is created fresh for each run and holds no real user data.

Code coverage is measured with pytest-cov. Each CI run prints a coverage report that lists, for every module in the backend's `app` package, the percentage of lines executed and the line numbers that were missed. Reviewers can use this report to check that a pull request tests the code it adds.

No minimum coverage is enforced at this stage. Coverage results will be summarized in the VnV Report.

The frontend's unit tests will use Jest, which works directly with React Native, and will run in CI alongside the backend tests. Jest has built-in coverage reporting, so frontend coverage will be reported on each run and summarized in the VnV Report in the same way.

3.6.3 Secret Scanning

The repository is public, so an API key, password, or token is exposed as soon as it is pushed. Secrets are kept in a local `.env` file, which Git ignores, and gitleaks checks for any that slip into a commit at two points:

- **Before each commit:** a pre-commit hook, configured in `.pre-commit-config.yaml`, scans the staged changes and stops the commit if they contain a secret. Each member sets it up once per clone, as described in `CONTRIBUTING.md`.
- **In CI:** the `Secret scan` workflow in `.github/workflows/secret-scan.yml` scans the commits in every pull request and every push to `main`. This catches a secret even if the hook was skipped or never set up.

Both checks use gitleaks 8.30.1, so they apply the same detection rules. Any secret found is redacted in the output, so a CI run does not expose it again. A secret that reaches the repository is revoked and replaced, since removing it from the history does not make it private again.

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should

plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

Title for Test

1. test-id1

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

4.1.2 Area of Testing2

...

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Area of Testing1

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

4.2.2 Area of Testing2

...

4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

Input:

Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

Author Author. System requirements specification. https://github.com/..., 2019.

Team 1, Burn n' Earn. Hazard analysis for fitfolio. https://github.com/davislenover/BurnNEarn/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf, 2026a. Version 1.0, October 19, 2026.

Team 1, Burn n' Earn. Software requirements specification for fitfolio. https://github.com/davislenover/BurnNEarn/blob/main/pdfs/SRS-IEEE/SRS.pdf, 2026b. Version 1.1, October 27, 2026.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

Davis: My main pain point was how much ground this deliverable had to cover, and a lot of that came from our own earlier work. The SRS has 77 functional requirements, and each one needs at least one test. We rushed to get the SRS done and did not really check whether all of those requirements were needed, so that cost came back here. My share of Section 4.1 alone came to 57 tests. I resolved this by splitting 4.1 with Farid, using one test format across the whole section, and working through it one feature area at a time. The usability questionnaire had a similar problem. It grew long and elaborate, and I could see that justifying all of it would be hard. I resolved this by cutting it back to what each goal actually needs and limiting the study to the core loop. Next time I will check requirements more carefully before moving on, since every requirement we keep has to be tested later.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?